

```

channel="ibm_quantum_platform",
token=API_KEY,
overwrite=True
)
6. service = QiskitRuntimeService( )
7. backend = service.least_
   busy(operational=True,
   simulator=False)
8. print(f"--> Target QPU Selected:
   {backend.name}")
9. qc = QuantumCircuit(1, 1)
10. qc.x(0)
11. qc.measure(0, 0)

12. pm = generate_preset_pass_
   manager(backend=backend,
   optimization_level=1)
13. isa_circuit = pm.run(qc)

14. sampler = Sampler(mode=backend
15. job = sampler.run([isa_circuit],
   shots=1000)
16. print(f"--> Job Submitted! Job ID:
   {job.job_id( )}")
17. result = job.result( )
18. pub_result = result[0]
19. counts = pub_result.data.c.get_
   counts( )
20. print("\nReal Hardware Results:",
   counts)

```

Since the program *qiskit8.py* does not introduce any new quantum computing concepts and differs from *qiskit4.py* only in the IBM Quantum Platform-specific code required to execute the circuit on real hardware, I shall not explain it line by line in order to save space. However, notice that the lines of code used to create the qubit, apply the Pauli-X gate, and perform the measurement are identical to those in *qiskit4.py*.

Before executing *qiskit8.py*, we must first install the IBM Runtime package. After activating the virtual environment *qiskit_env*, execute the following command in the terminal: *pip install qiskit-ibm-runtime*. Next, replace the placeholder *API_KEY* = "PasteYourAPIKeyHere" in *qiskit8.py* with the API key obtained from

```

Using instance: open-instance, plan: open
--> Target QPU Selected: ibm_marrakesh
--> Job Submitted! Job ID: d9qco0gpdb6s73e3cmeg

Real Hardware Results: {'1': 946, '0': 54}

```

Figure 3: Terminal output obtained after executing *qiskit8.py* on an IBM quantum processor

your IBM Quantum Platform account. Simply paste your API key inside the double quotation marks. Once these steps are complete, execute the program on an actual IBM quantum processor by running the command *python qiskit8.py* from the terminal. The measurement statistics obtained from executing the program on a real IBM quantum computer are shown in Figure 3.

While executing *qiskit8.py*, you may notice a few warning messages before the actual program output. These are normal and indicate that Qiskit IBM Runtime is automatically selecting an available instance for your account. In our case, the open instance was selected. Since IBM's quantum processors are shared by users worldwide, Open Plan users may occasionally experience waiting times. Depending on processor availability, IBM automatically selects a suitable backend. In our case, the program was executed on *ibm_marrakesh*, one of IBM's latest Heron R2 superconducting quantum processors.

The measurement statistics obtained from the real quantum processor are $\{ '1': 946, '0': 54 \}$. In contrast, an ideal simulator always produces $\{ '1': 1000 \}$ because the Pauli-X gate deterministically transforms the state $|0\rangle$ into $|1\rangle$. The 54 incorrect outcomes clearly demonstrate that present-day NISQ-era quantum hardware is prone to errors arising from imperfections such as gate errors, decoherence, and measurement errors. Fortunately, as demonstrated earlier in this article, Qiskit's noise models allow us to imitate these hardware imperfections using a simulator.

Interestingly, the real quantum processor produced 0 in 54 out of 1000 shots, corresponding to an error rate of about 5.4%. This is remarkably close to the 5% Pauli-X error probability used in our original noisy simulation. However, we should not conclude that the actual quantum processor has an overall error rate of exactly 5.4%. Our simple noise model represents only one type of error, whereas a real quantum processor is affected by several sources of error, including gate errors, decoherence, and measurement errors. Nevertheless, this comparison gives us useful intuition about how a simple noise model can approximate the behaviour observed on real hardware.

A closer look at Figure 2 reveals several useful details about the IBM Quantum Platform. Besides providing options for managing API keys, the dashboard also displays the available quantum instance (*open-instance*) and the remaining free usage time, which in our case is 9 minutes and 10 seconds. It also maintains a record of recently executed quantum jobs, showing their execution status, the quantum processor used (*ibm_marrakesh*), and the execution time for each job.

Now, let us take a small digression and introduce two important notations: the positive X-basis state, denoted by $|+\rangle$ (read as 'ket plus'), and the negative X-basis state, denoted by $|-\rangle$ (read as 'ket minus'), as shown in Figure 4. These states are obtained by applying the Hadamard gate to the computational

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

Figure 4: The states $|+\rangle$ and $|-\rangle$